Multipath QUIC (MP-QUIC) vs MPTCP: An Architectural and Literature-Based Comparison

Rathi Kaushal Mukund
Roll No. 24110290

Pratik Kumar Dhakar
Roll No. 24110277

Krina Hitendrakumar Paghadar
Roll No. 24110169

More Swamini Sanjaykumar
Roll No. 24110211

I. Introduction

Modern devices such as smartphones with Wi-Fi and cellular radios, and multi-homed servers, can reach the network over several paths at the same time. Classical TCP and single-path QUIC, however, use one path per connection. A TCP connection is identified by its 4-tuple (source and destination IP address and port), so when a phone moves from Wi-Fi to cellular and its address changes, the connection breaks and the application has to reconnect.

Multipath transport lets a single connection use several paths at once. This gives (1) bandwidth aggregation, where a transfer uses the combined capacity of the paths; (2) smooth handover, where traffic moves from a weakening interface to another one without dropping the session; and (3) resilience, where loss or failure on one path is masked by using another.

Multipath TCP (MPTCP) was first specified as version 0 (RFC 6824) and revised as version 1 in RFC 8684 [1], and is part of the mainline Linux kernel from version 5.6. To work across existing networks it has to look like ordinary TCP to middleboxes, which shapes its design. QUIC [2], [3] is a newer transport that runs over UDP, integrates TLS 1.3, and carries independent streams. Its multipath extension, draft-ietf-quic-multipath-21 [4], allows a QUIC connection to use several paths simultaneously and is in the RFC Editor queue.

This report is a theoretical, architectural comparison of MPTCP and MP-QUIC, based on the specifications and on published evaluation studies. No software was implemented and no experiments were run. Our contributions are: (i) a side-by-side description of how the two protocols set up and manage paths, order data and recover from loss; (ii) a comparison table; (iii) a conceptual discussion of congestion control, fairness and scheduling; and (iv) a qualitative table of the behaviour expected from the architecture in common network scenarios, noting where a published study reports the same trend.

II. Problem Statement

The project compares MP-QUIC and MPTCP at the architectural level, in terms of how their designs affect throughput, latency and connection stability for applications such as file transfer, video streaming and VoIP. Measuring this for each application is left to future work.

Adding a second path does not always help. When two paths differ strongly, for example in round-trip time (RTT) or loss rate, data sent over both arrives out of order at the receiver. Because data must be delivered in order (over the whole connection in MPTCP, per stream in MP-QUIC), data from the faster path waits in the receive buffer for data still travelling on the slower path. If the buffer fills, the fast path is stalled; this is head-of-line (HOL) blocking, and the multipath connection can end up slower than the best single path. How MPTCP and MP-QUIC are structured determines how much this hurts, and this is the gap our study examines.

- **Inputs:** the protocol specifications [1]–[4], the coupled congestion control specification [5], and published MPTCP/MP-QUIC evaluation studies [6], [7].
- **Outputs:** a structured comparison of the two protocols' mechanisms, and a qualitative discussion of their expected behaviour with similar and different paths, with loss, and on path failure.
- **Scope:** in scope are path setup and management, data ordering and HOL blocking, acknowledgements and loss recovery, and congestion control and scheduling at a conceptual level. Out of scope are any implementation, kernel or protocol-stack changes, emulation, measurements, and per-application protocol recommendations, which require measurements.

III. Project Objectives

- **Path setup and management.** Compare how MPTCP (MP_CAPABLE, MP_JOIN, ADD_ADDR, REMOVE_ADDR, MP_PRIO) and MP-QUIC (multipath negotiated in the handshake, connection IDs, path validation, path status) establish, authenticate and manage paths.
- **Data ordering and HOL blocking.** Explain MPTCP's subflow and data sequence numbers (DSS mapping) and MP-QUIC's per-path packet numbers with stream offsets, and contrast connection-wide with stream-level HOL blocking.
- **Acknowledgements and loss recovery.** Compare MPTCP's subflow-level and data-level acknowledgements, retransmission and reinjection with MP-QUIC's per-path acknowledgements and never-reused packet numbers.
- **Congestion control and scheduling.** Explain, at a conceptual level, why independent congestion control on

each path is unfair at a shared bottleneck, how coupled congestion control (LIA) addresses this, and why the scheduler matters when paths differ.

Building a testbed and measuring the protocols was not part of the completed work (Sections VI and VII).

IV. Architecture and Design

This section describes how each protocol realises multipath transport, compares them, and discusses the main design trade-offs.

A. MPTCP (version 1)

MPTCP aims to look like ordinary TCP to both applications and the network. The application sees a single byte-stream socket, and each path is carried as a normal TCP flow (a subflow) with its own 4-tuple, TCP state, congestion window and RTT estimate [1].

- **Setup and path management.** The first subflow negotiates MP_CAPABLE during the TCP handshake and exchanges keys from which a connection token is derived. Further subflows run a new TCP handshake carrying MP_JOIN, which includes the token, nonces and an HMAC to authenticate the join. If a middlebox removes the MP_CAPABLE option, the connection continues as regular TCP. The client normally opens subflows; a server announces extra addresses with ADD_ADDR, withdraws them with REMOVE_ADDR, and MP_PRIO marks a subflow as backup.
- **Sequence numbers and acknowledgements.** Each subflow keeps its own contiguous subflow sequence numbers, so each path looks like a valid TCP to middleboxes. A connection-level data sequence number orders data across subflows, and the DSS option maps subflow ranges to data sequence numbers and carries the connection-level Data ACK. A subflow ACK only confirms delivery on one path; the Data ACK confirms that data reached the MPTCP layer at the receiver.
- **Retransmission and reinjection.** Loss on a subflow is recovered by that subflow's TCP mechanisms. Data stuck on a failed or very slow subflow can be reinjected on another subflow.

Because MPTCP presents one ordered byte stream, a gap caused by loss or delay on one subflow can hold back data that has already arrived on the others (connection-wide HOL blocking).

B. MP-QUIC (draft-ietf-quic-multipath-21)

QUIC [2], [3] runs in user space over UDP, integrates TLS 1.3 into its handshake, encrypts most of the transport header and control information, and multiplexes independent streams. Base QUIC supports connection migration, but only one path is active at a time. The multipath extension [4] lets several paths be used at once.

- **Setup and path management.** Multipath is negotiated through a transport parameter during the handshake, and each path has its own identifier and connection IDs. A new path is validated with PATH_CHALLENGE and PATH_RESPONSE before use, can be marked available or backup, and is closed with PATH_ABANDON.
- **Packet numbers and acknowledgements.** Each path has its own packet number space, and packet numbers are never reused. Lost data (a STREAM frame carrying stream ID and offset) is retransmitted in a new packet with a new number. This removes TCP's retransmission ambiguity and gives cleaner RTT samples. Acknowledgements are reported per path (PATH_ACK).
- **Streams and HOL blocking.** Data order is kept by stream ID and offset, independent of the path a packet took. A lost packet delays only the streams whose frames it carried, so HOL blocking is per stream rather than connection-wide.

C. Comparison

TABLE I. MPTCP v1 vs. MP-QUIC at a glance

Aspect	MPTCP v1	MP-QUIC (draft-21)
Layer	Kernel TCP extension; control information in TCP options.	User space over UDP; control frames inside encrypted payloads.
Path setup	MP_CAPABLE, then MP_JOIN per subflow (token, nonces, HMAC).	Negotiated in handshake; new path validated with PATH_CHALLENGE / PATH_RESPONSE.
Path management	ADD_ADDR, REMOVE_ADDR, MP_PRIO.	Path status (available / backup) and PATH_ABANDON.
Ordering	Subflow sequence number + data sequence number (DSS).	Per-path packet numbers + stream ID and offset.
HOL scope	Connection-wide.	Per stream.
ACK / loss	Subflow ACK + Data ACK; subflow retransmission, reinjection.	Per-path ACKs; lost frames resent in new packets.
Congestion control	Per-subflow windows; coupled variants (LIA).	Separate state per path [4]; the draft notes LIA [5] can be adapted.
Middleboxes	TCP options may be stripped, causing fallback to TCP.	Not exposed to TCP-option changes; UDP may be blocked or rate-limited.

D. Congestion Control and Scheduling

If every path runs its own congestion control and the paths share a bottleneck, a two-path connection takes about twice the share of a single-path flow, which is unfair to other users. MPTCP addresses this with coupled congestion control (LIA [5] and later variants), which links the subflow windows so that the connection as a whole is no more aggressive than a single TCP flow on its best path while still moving traffic towards less congested paths. In MP-QUIC the draft requires separate congestion control state for each path and notes that LIA can be adapted to it, because standard congestion control on paths that share a bottleneck can be unfair [4].

The scheduler decides which data is sent on which path. A simple lowest-RTT-first policy can backfire when RTTs differ strongly, because data arriving late from the slow path fills the receive buffer and stalls the fast path. More elaborate

schedulers try to take this into account. We studied the role of scheduling conceptually and did not study individual schedulers.

E. Design Trade-offs

MPTCP's choices (TCP options, two sequence spaces, fallback to TCP) buy compatibility with unmodified applications and existing middleboxes, at the cost of added complexity and connection-wide HOL blocking. MP-QUIC's choices (UDP, encryption, streams, per-path packet numbers) give cleaner per-path loss detection and stream-level HOL isolation, but depend on UDP being allowed, on user-space library support, and on a specification that is still evolving. Single-path QUIC with connection migration is a simpler alternative, but it uses one path at a time and therefore offers no aggregation.

V. Implementation

No software was implemented in this project: there were no kernel changes, protocol code or custom tools, and so no languages, frameworks or hardware to report. The work is theoretical and architectural. The topics were divided among the team members (Appendix A) and combined into the single comparison presented here. The main challenge was that the specifications are long and technical and the MP-QUIC specification is still evolving; we therefore worked from the latest draft revision (-21) [4].

Repository: https://github.com/Krina-Paghadar/CS331-T007-MPTCP-vs-MPQUIC (no implementation code).

VI. Testbed Setup

No testbed was built and no experiments were run, so there is no hardware, operating system, network topology, workload or configuration to report. The emulation tools needed for such experiments (for example Mininet and NetEm) were not set up and are part of future work.

For context, the published studies we refer to used two kinds of setup: emulated network topologies with controlled delay, bandwidth and loss [6], and real smartphones comparing MPTCP and MP-QUIC [7]. A future testbed of ours would follow the same two approaches.

VII. Evaluation

We performed no experiments, so this section contains no measurements of our own. Table II summarises the behaviour expected from the architecture in Section IV; entries marked † are also reported by a published study [6].

TABLE II. Qualitative comparison (no measurements)

Scenario	MPTCP v1	MP-QUIC (draft-21)
Similar paths	Can aggregate the capacity of both paths.	Can aggregate both paths; keeps MPTCP's aggregation benefit †.
No packet loss	Compared against MP-QUIC in the published study †.	Reported to outperform MPTCP in some cases †.
Loss on a path	Loss on one subflow can stall the whole ordered byte stream.	Loss delays only the affected streams; QUIC-based protocols reported more suitable under loss †.
Very different paths	Reordering can fill the receive buffer and stall the fast path for the whole connection.	Same reordering effect, but ordering is per stream, so streams not waiting on the slow path are less affected.
Path failure or handover	Continues on remaining subflows; stuck data can be reinjected.	Continues on remaining paths; lost frames are resent in new packets; failed path is closed with PATH_ABANDON.
Restrictive networks	TCP options may be stripped; falls back to single-path TCP.	Not exposed to TCP-option changes; affected if UDP is blocked or rate-limited.

† Reported in the published study [6]. Other entries follow from the architecture and have not been measured by us. A separate study on smartphones [7] found no clear winner for handover when both protocols used the same Wi-Fi-first scheduling strategy, but MPTCP reached higher throughput when upload bandwidth exceeded 50 Mbps, which its authors consider probably due to implementation overhead.

A. Application Scenarios (Conceptual)

Bulk file transfer benefits mainly from aggregation. Video streaming is sensitive to stalls, so a slow path delaying data can cause rebuffering. VoIP is sensitive to delay and jitter, so a second path mainly helps resilience and handover, provided the slower path does not delay packets. Which protocol and scheduler is best for each application can only be decided with measurements, which we did not perform, so we make no per-application recommendation.

B. Caveat

The published studies [6], [7] used earlier implementations and an earlier MP-QUIC design than the current draft. Their results are therefore useful for the qualitative trends they show, not as benchmarks of the protocols as they exist today.

VIII. Summary

We compared MPTCP v1 and MP-QUIC at the architectural level. MPTCP stays compatible with TCP applications and middleboxes through TCP options and two sequence spaces, but its single ordered byte stream causes connection-wide HOL blocking. MP-QUIC uses encrypted headers, per-path packet numbers and independent streams, which limit HOL blocking to individual streams, but it depends on UDP being usable and on a specification that is still evolving. A published study [6] reports that MP-QUIC keeps the aggregation and handover benefits of MPTCP and that QUIC-based protocols are more suitable under loss, while a smartphone study [7] found no clear winner for handover. Overall, we expect behaviour to depend on how similar the paths are, on loss, and on the workload.

Limitations: the study is theoretical, with no experiments of our own; the published results come from earlier

implementations; the MP-QUIC specification may still change; and we make no per-application recommendation. Future work is to set up an emulation testbed (for example Mininet and NetEm) and measure current MPTCP and MP-QUIC implementations with similar and different paths, with loss and path failure, and with file transfer, video and VoIP workloads, and to study coupled congestion control and schedulers in more depth.

IX. Reproducibility

Repository: https://github.com/Krina-Paghadar/CS331-T007-MPTCP-vs-MPQUIC. This project is a theory and literature study: no code was written, no testbed was built and no experiments were executed, so there are no scripts or commands to reproduce. The statements in this report can be checked against the specifications [1]–[5] and the published studies [6], [7].

References

[1] A. Ford, C. Raiciu, M. Handley, O. Bonaventure, and C. Paasch, "TCP Extensions for Multipath Operation with Multiple Addresses," IETF RFC 8684, Mar. 2020, doi: 10.17487/RFC8684.

[2] J. Iyengar and M. Thomson, "QUIC: A UDP-Based Multiplexed and Secure Transport," IETF RFC 9000, May 2021, doi: 10.17487/RFC9000.

[3] J. Iyengar and I. Swett, "QUIC Loss Detection and Congestion Control," IETF RFC 9002, May 2021, doi: 10.17487/RFC9002.

[4] Y. Liu, Y. Ma, Q. De Coninck, O. Bonaventure, C. Huitema, and M. Kühlewind, "Managing Multiple Paths for a QUIC Connection," IETF Internet-Draft draft-ietf-quic-multipath-21 (work in progress), Mar. 2026.

[5] C. Raiciu, M. Handley, and D. Wischik, "Coupled Congestion Control for Multipath Transport Protocols," IETF RFC 6356, Oct. 2011, doi: 10.17487/RFC6356.

[6] Q. De Coninck and O. Bonaventure, "Multipath QUIC: Design and Evaluation," in Proc. ACM CoNEXT '17, Incheon, Republic of Korea, Dec. 2017, pp. 160–166, doi: 10.1145/3143361.3143370.

[7] Q. De Coninck and O. Bonaventure, "MultipathTester: Comparing MPTCP and MPQUIC in Mobile Environments," in Proc. 3rd Workshop on Mobile Network Measurement (MNM '19), co-located with the Network Traffic Measurement and Analysis (TMA) Conf., Paris, France, Jun. 2019, pp. 221–226.

Appendix A: Team Work Distribution and Member Rating

A.1 Work Distribution

#	Member Name	Roll No.	Tasks Done	Rating (1–10, unique)
1	Rathi Kaushal Mukund	24110290	multipath QUIC, path IDs and negotiation, one packet number space per path, opening and closing paths (PATH_CHALLENGE/PATH_RESPONSE, PATH_ABANDON), path status, and per-path acknowledgements and congestion control.	10
2	Krina Hitendrakumar Paghadar	24110169	MPTCP architecture and connection establishment (MP_CAPABLE, MP_JOIN); TCP sequence numbers vs. data sequence numbers and DSS mappings; ACK mechanisms; retransmission and reinjection; path management (ADD_ADDR, REMOVE_ADDR, MP_PRIO). Conceptual roles of scheduling and coupled congestion control.	9.5
3	More Swamini Sanjaykumar	24110211	Multipath fundamentals, protocol history, application scenarios and QoE implications; conceptual aspects of congestion control and scheduling. Reviewed published evaluation studies of MPTCP and MP-QUIC for their key findings and testbed setups.	9
4	Pratik Kumar Dhakar	24110277	Coupled congestion control (LIA) and the conceptual aspects of fairness, scheduling and congestion control in multipath transport. Transport-layer foundations.	8

Appendix B: AI Usage Details

B.1 Prompts

#	AI Tool	Purpose
1	Gemini	Report making: drafted the initial structure and text of the report, which the team then revised.
2	Claude (Anthropic)	Condensed study plan and a question bank used to divide topics across the team; explanations of MPTCP mechanisms; guidance on repository structure; revision of the report draft, including reformatting it to the required template and aligning its claims with the completed work, and checking the report and slides against the cited sources.
3	ChatGPT (OpenAI)	Early-phase help in interpreting the intended project scope.

Relevant prompt queries and response .md files will be added to the AI_Used folder of the GitHub repository before the deadline.